

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕ-
ЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
“ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ”
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук

Кафедра программирования и информационных технологий

Создание мобильного приложения «Roadmap»

Курсовая работа

09.03.02 Информационные системы и технологии

Профиль «Программная инженерия в информационных системах»

Зав. кафедрой _____ д. ф-м.н., профессор С.Д. Махортов

Обучающийся _____ ст. 3 курса оч. отд. В.Д. Труфанов

Руководитель _____ ст. преподаватель В.С. Тарасов

Консультант _____ ассистент В.А. Ушаков

Воронеж 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....	4
ВВЕДЕНИЕ	6
1 Обзор предметной области	9
1.1 Описание предметной области	9
1.2 Общее описание рынка образовательных IT - приложений.....	10
1.3 Обзор аналогов.....	10
1.3.1 Roadmap.sh.....	11
1.3.2 Coursera	12
1.3.3 Хеклет	13
1.3.4 Результат сравнения аналогов	14
2 Проектирование системы	16
2.1 Пользовательские сценарии	16
2.2 Выбор стека технологий	17
2.3 Архитектура разрабатываемой системы	19
2.3.1 Серверная часть приложения.....	19
2.3.2 База данных.....	22
2.3.3 Клиентская часть приложения	24
2.3.4 Взаимодействие клиент-сервер.....	25
3 Реализация разрабатываемой системы	26
3.1 Серверная часть приложения.....	26
3.2 Клиентская часть приложения	27
3.2.1 Экран загрузки и приветствия.....	27
3.2.2 Экран авторизации и регистрации	29
3.2.3 Главный экран	31
3.2.4 Экран каталога Дорожных карт	32
3.2.5 Экран дорожной карты.....	33
3.2.6 Экран профиля	34

Заключение	37
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	39

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей работе применяются следующие термины с соответствующими определениями:

Эндпоинт (endpoint) — конечная точка доступа к программному интерфейсу, представленная определённым URL-адресом, к которому клиент обращается для выполнения операции.

Токен — уникальный набор данных, выдаваемый сервером после успешной аутентификации и используемый клиентом для подтверждения своих прав при последующих запросах.

Корутина — механизм языка Kotlin для написания асинхронного кода в последовательном стиле без блокировки потоков выполнения.

API — (Application Programming Interface) программный интерфейс приложения — набор правил и механизмов взаимодействия между программными компонентами.

REST — (Representational State Transfer) архитектурный стиль построения распределённых приложений, основанный на стандартных методах протокола HTTP.

HTTP — (HyperText Transfer Protocol) протокол передачи данных в сети, лежащий в основе обмена информацией в сети Интернет.

JSON — (JavaScript Object Notation) текстовый формат обмена данными, основанный на представлении объектов в виде пар «ключ — значение».

JWT — (JSON Web Token) открытый стандарт создания токенов доступа, используемый для аутентификации и авторизации пользователей.

СУБД — система управления базами данных — программное обеспечение для создания, ведения и совместного использования баз данных.

ORM — (Object-Relational Mapping) объектно-реляционное отображение — технология, связывающая объекты программы с записями реляционной базы данных.

SQL — (Structured Query Language) язык структурированных запросов для работы с реляционными базами данных.

MVVM — (Model-View-ViewModel) архитектурный паттерн, разделяющий приложение на три слоя: модель данных, представление и модель представления.

UI — (User Interface) пользовательский интерфейс — совокупность средств взаимодействия пользователя с программой.

IDE — (Integrated Development Environment) интегрированная среда разработки программного обеспечения.

JDBC — (Java Database Connectivity) программный интерфейс для подключения и выполнения запросов к базам данных из приложений на платформе JVM.

CRUD — (Create, Read, Update, Delete) базовый набор операций над данными: создание, чтение, обновление и удаление.

ER-диаграмма — (Entity-Relationship Diagram) диаграмма «сущность — связь», используемая для проектирования структуры базы данных.

SDK — (Software Development Kit) комплект средств разработки программного обеспечения для определённой платформы.

ВВЕДЕНИЕ

Сфера информационных технологий является одной из наиболее динамично развивающихся областей современной экономики. Постоянное появление новых языков программирования, фреймворков, инструментов и методологий ставит перед начинающими и практикующими специалистами сложную задачу: определить, какие именно знания и навыки необходимо освоить для построения карьеры в выбранном направлении и в какой последовательности это следует делать.

Начинающий разработчик, желающий стать, например, frontend-инженером или специалистом по DevOps, сталкивается с колоссальным объёмом разрозненной информации. Бесчисленное количество курсов, статей, видеоуроков и книг создаёт информационный шум, в котором крайне трудно выстроить целостную и логически выверенную траекторию обучения. Именно эту проблему призваны решать так называемые дорожные карты обучения (learning roadmaps) — структурированные визуальные схемы, отображающие последовательность тем и технологий, которые необходимо изучить для овладения той или иной профессией или навыком.

Концепция дорожных карт обучения получила широкое признание благодаря популярному веб-проекту roadmap.sh, который на сегодняшний день является одним из наиболее популярных репозиторий по количеству звезд на платформе GitHub и ежемесячно посещается сотнями тысяч разработчиков по всему миру. Однако данный проект существует преимущественно в формате веб-сайта, тогда как современный пользователь всё чаще предпочитает потреблять контент и взаимодействовать с сервисами с мобильных устройств.

Разрабатываемое в рамках настоящей курсовой работы мобильное приложение «Roadmap» представляет собой Android-платформу, предоставляющую пользователям доступ к структурированным дорожным картам обучения по различным IT-профессиям (Frontend, Backend, DevOps, Android-разработка

и др.) и отдельным навыкам (Python, React, SQL, Docker и др.). Приложение позволяет не только просматривать дорожные карты в виде интерактивного графа взаимосвязанных тем, но и отмечать изученные темы, отслеживать персональный прогресс обучения и получать доступ к подобранным образовательным ресурсам по каждой теме.

Целью данной курсовой работы является проектирование и разработка мобильного приложения «Roadmap», обеспечивающего удобный доступ к дорожным картам обучения по IT-профессиям и навыкам, а также позволяющего пользователю отслеживать собственный прогресс в их освоении.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ предметной области и изучить существующие аналоги сервисов с дорожными картами и образовательных платформ;
- спроектировать архитектуру системы;
- выбрать технологический стек для реализации приложения;
- реализовать серверную часть приложения, обеспечивающую REST API для работы с дорожными картами и прогрессом пользователей;
- разработать клиентское Android-приложение с полным набором пользовательских экранов, включая интерактивную визуализацию дорожной карты;

Объектом исследования является процесс структурированного обучения IT-специалистов. Предметом исследования выступают методы и инструменты программной реализации мобильного приложения для предоставления дорожных карт обучения на платформе Android.

Практическая значимость работы состоит в создании готового к использованию мобильного приложения, которое поможет начинающим и практику-

ющим IT-специалистам выстроить осознанную и последовательную траекторию обучения, сэкономив время на поиске и систематизации учебных материалов.

1 Обзор предметной области

1.1 Описание предметной области

Предметной областью разрабатываемой системы является дорожная карта обучения — структурированное представление последовательности тем, технологий и навыков, которые необходимо освоить для овладения определённой IT-профессией или отдельным техническим навыком. В отличие от классических образовательных платформ, предлагающих отдельные курсы, дорожная карта ориентирована на формирование у пользователя целостного представления о траектории обучения и взаимосвязях между изучаемыми темами.

Дорожные карты обучения можно классифицировать по нескольким критериям: по типу (карты, основанные на профессиях, и карты, основанные на отдельных навыках), по способу представления (текстовые списки, ментальные карты, интерактивные графы) и по наличию функций отслеживания прогресса (пассивный просмотр или активное вовлечение с фиксацией изученного). Схема классификации представлена на рисунке 1

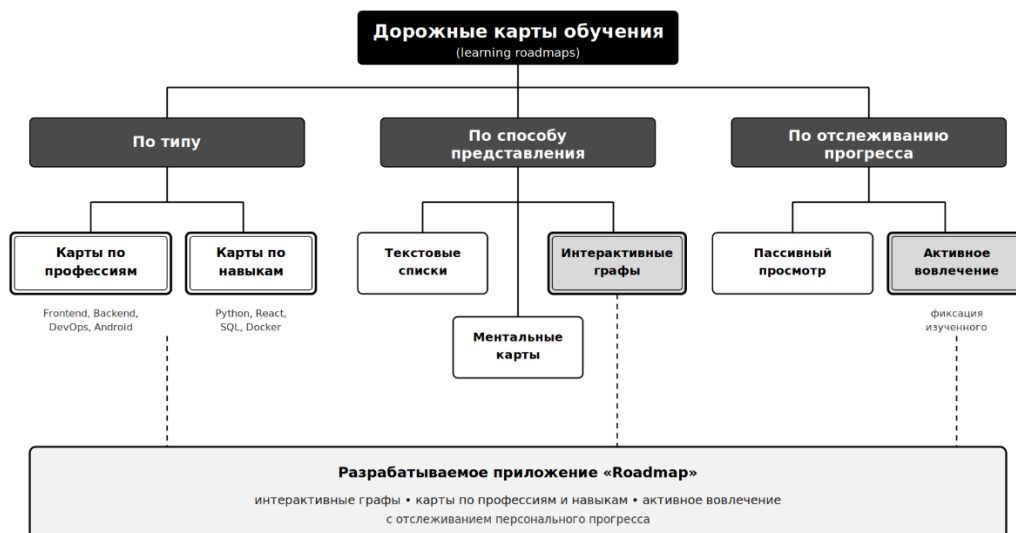


Рисунок 1 - Классификация дорожных карт обучения

Разрабатываемое приложение «Roadmap» относится к категории интерактивных дорожных карт с визуализацией в виде графа и поддержкой отслеживания персонального прогресса, объединяющих в себе карты по профессиям и карты по навыкам.

Таким образом, разработка приложения с интерактивными дорожными картами, наглядной визуализацией последовательности обучения и системой отслеживания прогресса отвечает как на запросы начинающих специалистов, так и на потребности практикующих разработчиков, осваивающих новые направления.

1.2 Общее описание рынка образовательных IT - приложений

Следует отметить, что большинство существующих решений либо представляют собой каталоги отдельных курсов без целостного представления о последовательности обучения, либо существуют преимущественно в веб-формате и слабо адаптированы под мобильное использование. Ниша мобильных приложений с интерактивными дорожными картами обучения, объединяющих наглядную визуализацию траектории и отслеживание персонального прогресса, на сегодняшний день заполнена недостаточно.

1.3 Обзор аналогов

Для выявления сильных и слабых сторон существующих решений были определены общие критерии сравнения:

- наличие структурированных дорожных карт по профессиям и навыкам;
- визуализация последовательности обучения в виде графа или схемы;

- наличие нативного мобильного приложения для платформы Android;
- наличие системы отслеживания персонального прогресса обучения.
-

1.3.1 Roadmap.sh

roadmap.sh — поддерживаемый сообществом веб-проект, предоставляющий дорожные карты, руководства и образовательные материалы для разработчиков. Проект является одним из наиболее популярных репозиторий на платформе GitHub и насчитывает миллионы зарегистрированных пользователей. Сервис предлагает дорожные карты двух типов: карты, основанные на ролях (Frontend, Backend, DevOps и др.), и карты, основанные на навыках (React, Python, SQL и др.). Каждая карта представлена в виде интерактивного графа, узлы которого соответствуют темам, а связи отражают рекомендуемую последовательность изучения. Проект существует преимущественно в формате веб-сайта; нативное мобильное приложение для Android отсутствует, а просмотр объёмных карт в мобильном браузере затруднён. Иллюстрация главной страницы сервиса представлена на рисунке 2. [1]

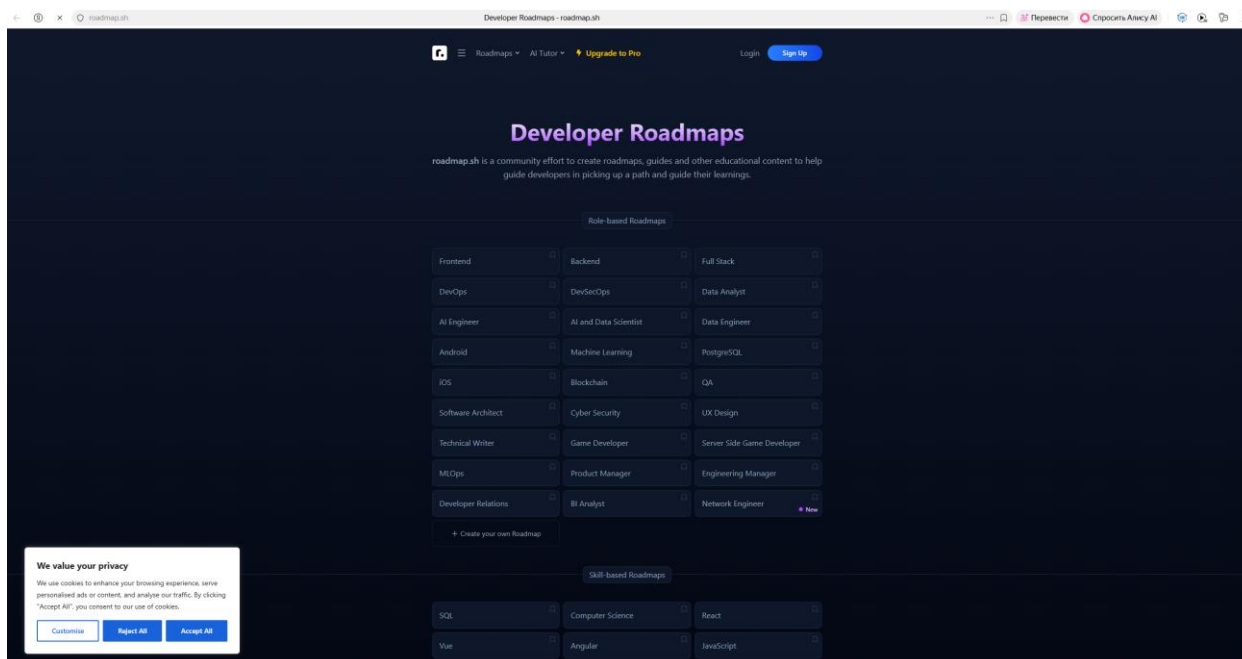


Рисунок 2 - Главная страница сервиса roadmap.sh

1.3.2 Coursera

Coursera — одна из крупнейших в мире платформ массовых открытых онлайн-курсов (МООС), сотрудничающая с ведущими университетами и компаниями. Платформа предлагает тысячи отдельных курсов и специализаций по широкому спектру дисциплин, включая значительный блок IT-направлений, и располагает развитым мобильным приложением с поддержкой просмотра лекций, в том числе в режиме офлайн. Однако Coursera предоставляет каталог отдельных курсов и не даёт пользователю целостной визуальной картины траектории становления в профессии; взаимосвязи между темами и рекомендуемая последовательность их изучения остаются неочевидными. Иллюстрация главного экрана приложения представлена на рисунке 3. [2]

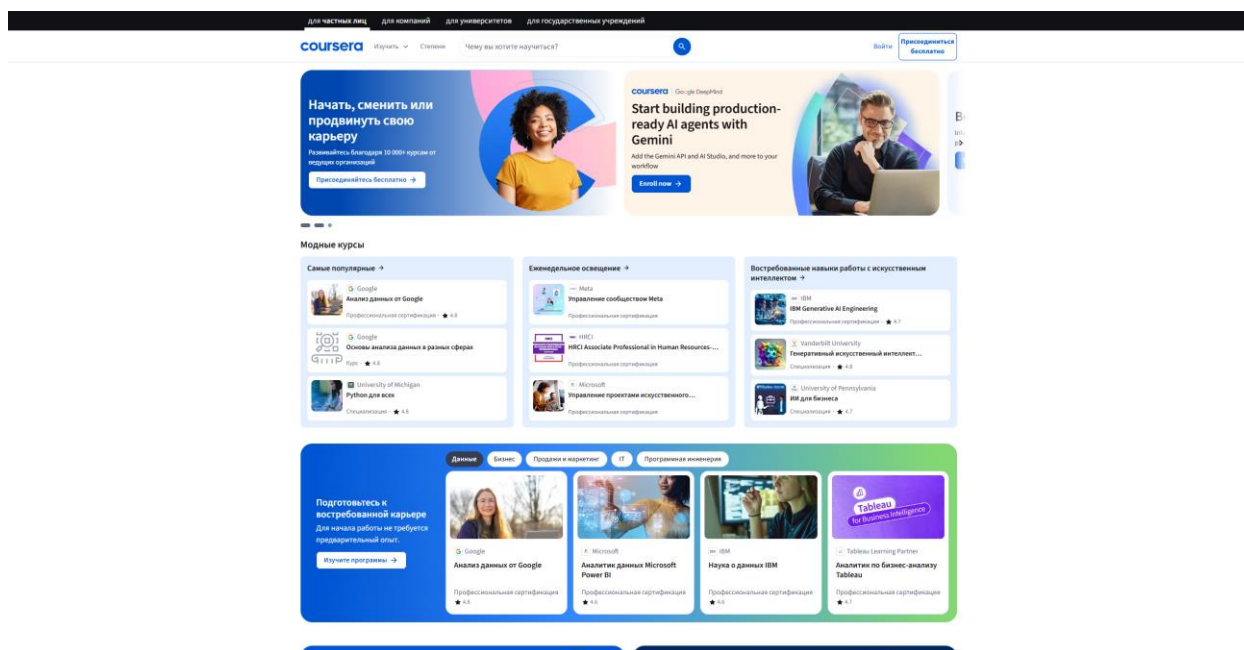


Рисунок 3 - Главный экран приложения Coursera

1.3.3 Хекслет

Хекслет — российская образовательная платформа для практического обучения программированию. Платформа предлагает структурированные программы обучения, объединённые в так называемые профессии, что в определённой степени напоминает концепцию дорожной карты, и делает акцент на практике с выполнением заданий в интерактивной среде. Вместе с тем платформа привязывает обучающегося исключительно к собственным учебным материалам, большая часть контента доступна только по платной подписке, а количество направлений существенно меньше разнообразия дорожных карт roadmap.sh. Наглядная визуализация траектории обучения в виде графа в платформе отсутствует. Иллюстрация главного экрана приложения представлена на рисунке 4. [3]

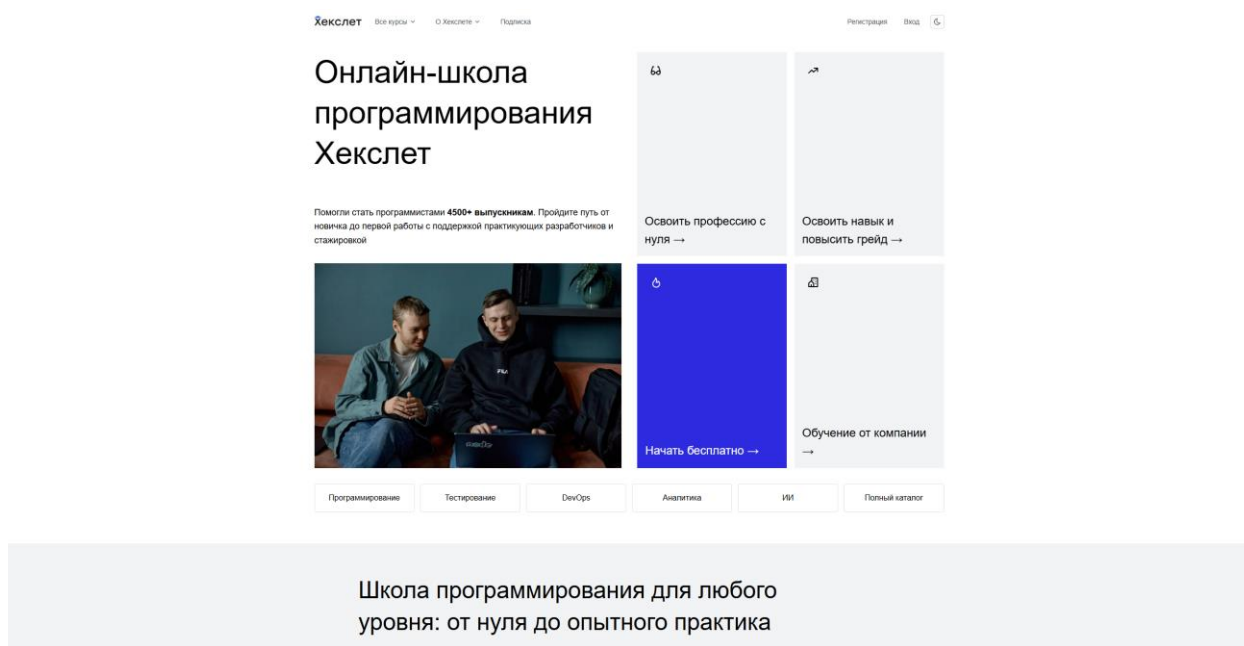


Рисунок 4 - Главный экран платформы Хекслет

1.3.4 Результат сравнения аналогов

Результаты сравнения функциональных возможностей аналогов представлены в таблице 1.

Приложение	Карты по профессиям и навыкам	Визуализация в виде графа	Нативное Android-приложение	Отслеживание прогресса
roadmap.sh	+	+	—	±
Coursera	—	—	+	+
Хекслет	±	—	±	+
Roadmap	+	+	+	+

Таблица 1 – Сравнение функциональных возможностей аналогов

Анализ показал, что ни один из рассмотренных аналогов не сочетает структурированные дорожные карты по профессиям и навыкам, наглядную визуализацию траектории обучения в виде интерактивного графа, нативное

Android-приложение и полноценное отслеживание персонального прогресса в едином решении. Выявленные функциональные пробелы определяют конкурентные преимущества разрабатываемой системы «Roadmap».

- По результатам анализа аналогов был сформирован следующий перечень ключевых функций приложения:
- просмотр каталога дорожных карт, разделённого на карты по профессиям и карты по навыкам;
- поиск дорожных карт по названию;
- просмотр выбранной дорожной карты в виде интерактивного графа взаимосвязанных тем;
- просмотр детальной информации о теме через нижний лист (bottom sheet);
- отметка темы как изученной, находящейся в процессе изучения или пропущенной;
- отображение статистики и прогресса прохождения дорожных карт;
- управление профилем пользователя;
- регистрация и авторизация в системе.

2 Проектирование системы

2.1 Пользовательские сценарии

Для определения основных функций мобильного приложения были сформированы пользовательские сценарии с помощью диаграммы прецедентов. Спроектированная диаграмма прецедентов представлена на рисунке 5.

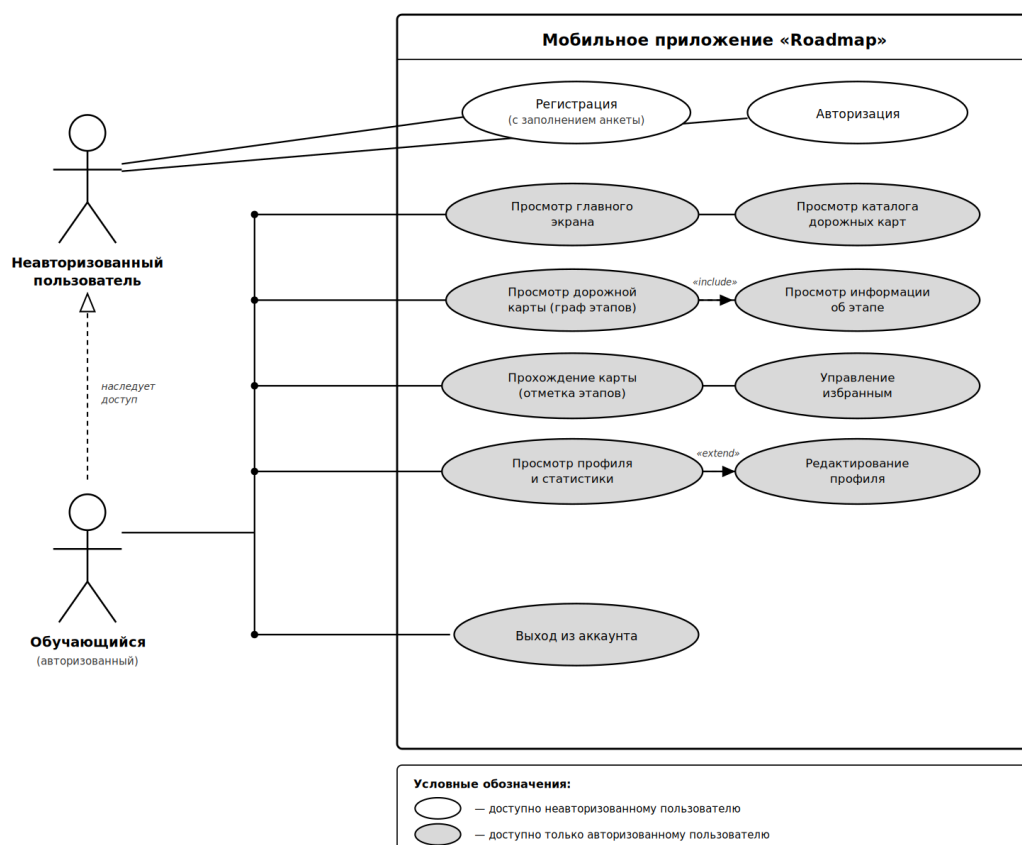


Рисунок 5 - Диаграмма прецедентов

В системе предусмотрен один тип пользователя — «Обучающийся» (авторизованный пользователь приложения). Неавторизованный пользователь имеет доступ только к экранам приветствия, входа и регистрации. Для работы в приложении пользователю необходимо пройти регистрацию, указав адрес

электронной почты и пароль, после чего заполнить анкету профиля. При последующих запусках осуществляется автоматическая авторизация по сохранённому токену.

Основные пользовательские сценарии приложения:

- регистрация в системе с последующим заполнением анкеты профиля (имя, фамилия, дата рождения);
- авторизация по адресу электронной почты и паролю;
- просмотр главного экрана с рекомендованными и избранными дорожными картами;
- просмотр каталога всех доступных дорожных карт;
- выбор дорожной карты и просмотр её в виде графа этапов (milestones);
- просмотр детальной информации об этапе и его описания;
- начало прохождения дорожной карты и отметка этапов как выполненных;
- добавление дорожной карты в избранное;
- просмотр профиля со статистикой прогресса и списком избранных карт;
- редактирование данных профиля (имя, фамилия, дата рождения, аватар);
- выход из аккаунта.

2.2 Выбор стека технологий

Для реализации клиентской (Android) части приложения выбраны инструменты и технологии, представленные ниже.

Язык программирования Kotlin — современный статически типизированный язык программирования для JVM-платформы, официально рекомендо-

ванный Google для разработки Android-приложений. Kotlin обеспечивает лаконичность кода, безопасность в отношении нулевых ссылок и полную совместимость с Java; [4]

Jetpack Compose — современный декларативный UI-фреймворк для Android от Google. Позволяет описывать пользовательский интерфейс в виде функций-компонентов на Kotlin, использует Material Design 3 в качестве дизайн-системы и обеспечивает реактивное обновление интерфейса при изменении состояния; [5]

Архитектурный паттерн MVVM (Model–View–ViewModel) — разделяет логику представления и бизнес-логику. ViewModel хранит состояние экрана и предоставляет UI-компонентам необходимые данные через StateFlow, что повышает тестируемость и масштабируемость кода; [6]

Canvas API — низкоуровневый программный интерфейс отрисовки графики в Android, использованный для визуализации дорожной карты в виде последовательности этапов (milestones); [7]

Для реализации серверной части приложения выбраны средства, описанные ниже.

Язык программирования Kotlin — используется как на клиентской, так и на серверной стороне, что обеспечивает единообразие кодовой базы и удобство совместного использования моделей данных; [8]

Фреймворк Ktor — лёгкий асинхронный фреймворк для создания серверных приложений на Kotlin от JetBrains. Ktor построен на корутинах Kotlin и обеспечивает высокую производительность при минимальных накладных расходах; [9]

СУБД PostgreSQL — реляционная система управления базами данных с открытым исходным кодом, отличающаяся высокой надёжностью и развитыми механизмами обеспечения целостности данных, что важно для хранения связанных структур дорожных карт и их этапов; [10]

ORM Exposed — лёгкая библиотека для работы с базами данных на Kotlin от JetBrains. Предоставляет DSL для построения запросов и маппинг данных между объектами Kotlin и таблицами базы данных; [11]

BCrypt — алгоритм хэширования паролей, обеспечивающий безопасное хранение паролей пользователей в базе данных.

2.3 Архитектура разрабатываемой системы

Мобильное приложение «Roadmap» построено на архитектуре клиент-серверного взаимодействия. Android-приложение выступает в роли клиента и взаимодействует с серверной частью через REST API по протоколу HTTP.

2.3.1 Серверная часть приложения

Серверная часть приложения реализована на фреймворке Ktor и организована по принципу маршрутизации запросов (Routing). Сервер предоставляет набор REST API эндпоинтов, сгруппированных по функциональным модулям. Структура серверной части представлена на рисунке 6.

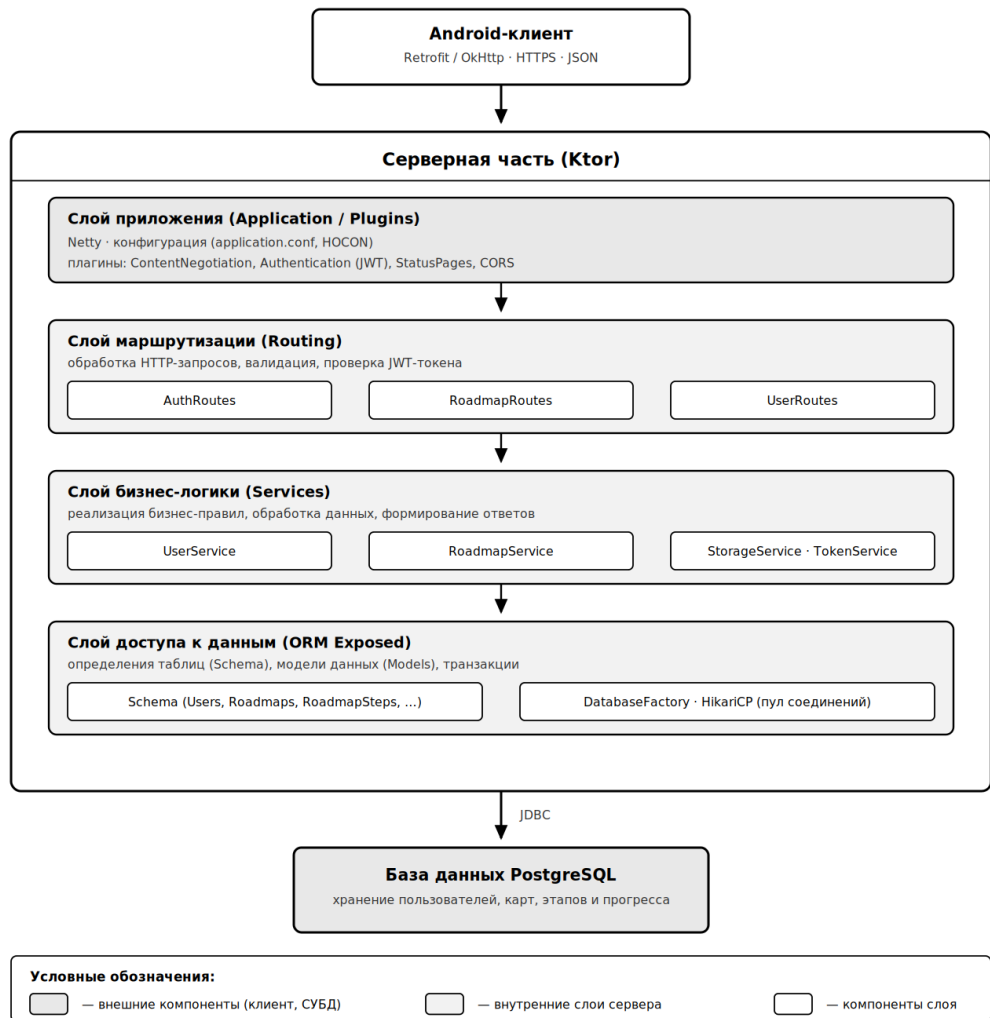


Рисунок 6 - Многоуровневая архитектура серверной части

Серверное приложение включает следующие группы маршрутов:

- authRoutes — маршруты для регистрации и авторизации пользователей (POST /auth/register, POST /auth/login);
- userRoutes — маршруты для управления профилем пользователя и загрузки аватара (GET и PUT /profile, POST /profile/upload);
- roadmapRoutes — маршруты для работы с дорожными картами: получение карты (GET /roadmaps/{id}), получение её этапов (GET /roadmaps/{id}/steps), работа с избранным (GET /roadmaps/favorites, POST /roadmaps/{id}/favorite), получение прогресса (GET /roadmaps/progress, GET /roadmaps/{id}/progress),

начало прохождения карты (POST /roadmaps/{id}/start) и отметка этапа выполненным (POST /roadmaps/steps/{stepId}/complete).

Безопасность API обеспечивается посредством JWT-аутентификации. При входе пользователю выдаётся JWT-токен, который клиент прикладывает к каждому последующему запросу в заголовке Authorization. Все маршруты, кроме регистрации и авторизации, обёрнуты в блок authenticate и доступны только при наличии валидного токена. Пароли пользователей хранятся в базе данных в хэшированном виде с применением алгоритма BCrypt. Схема эндпоинтов API представлена на рисунке 7. [12]

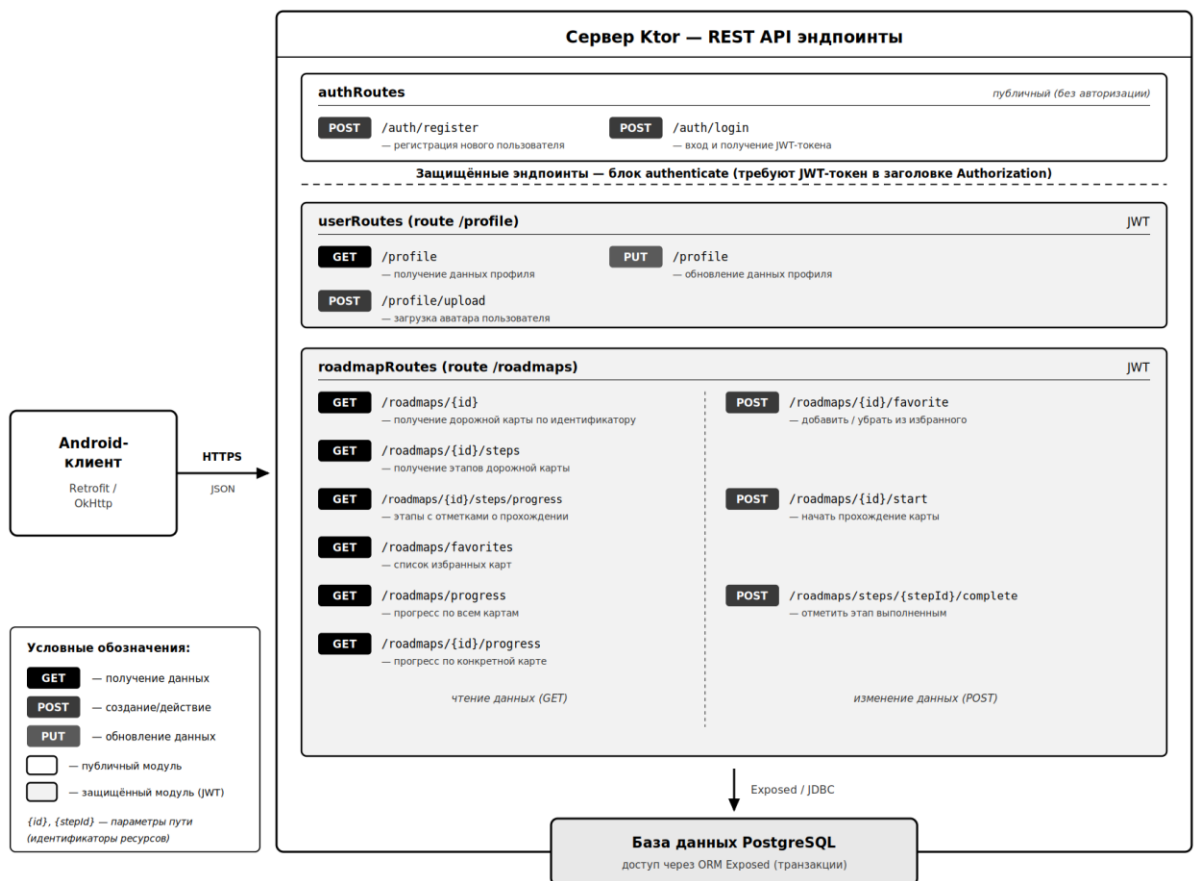


Рисунок 7 - Схема REST API эндпоинтов приложения

2.3.2 База данных

Для хранения данных используется реляционная СУБД PostgreSQL. База данных спроектирована с приведением к третьей нормальной форме и отражает структуру дорожных карт, состоящих из последовательности этапов, а также данные о пользователях и их прогрессе. Схема включает восемь таблиц, связанных между собой посредством внешних ключей. ER-диаграмма базы данных представлена на рисунке 8.

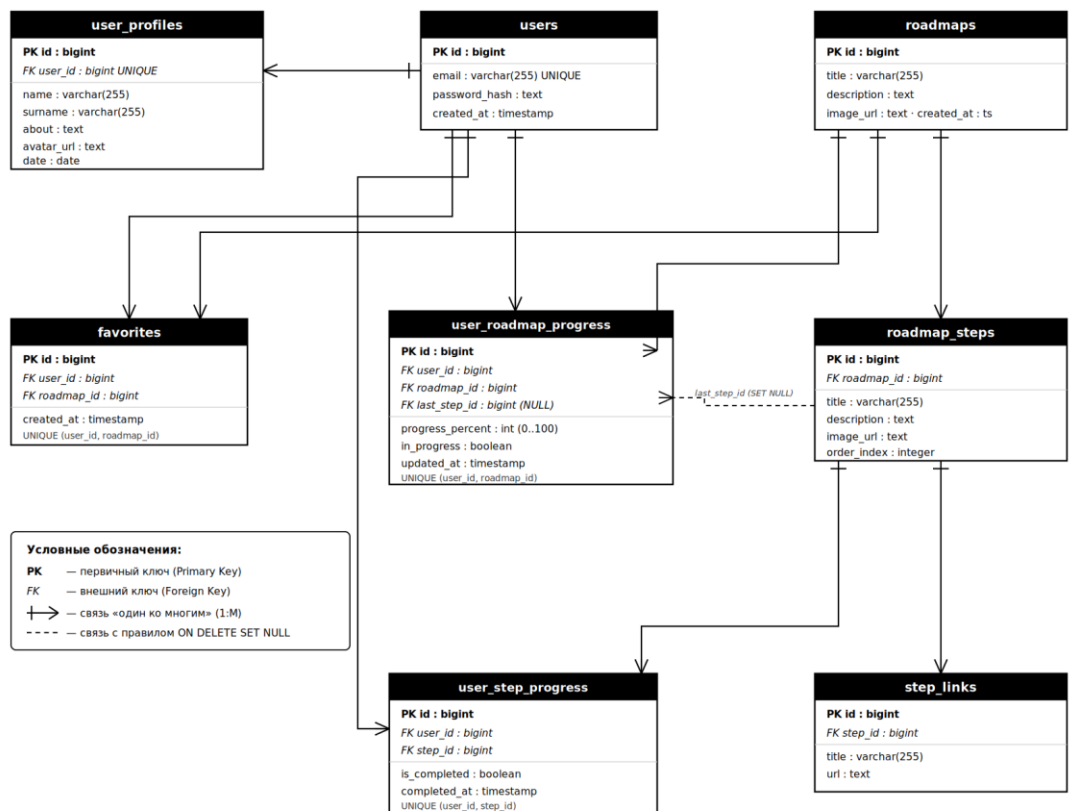


Рисунок 8 - ER-диаграмма базы данных

База данных включает следующие таблицы:

- users — данные о зарегистрированных пользователях: id (первичный ключ), email (уникальный адрес электронной почты), password_hash (хэш пароля), created_at (дата и время регистрации);
- user_profiles — анкетные данные пользователя: id (первичный ключ), user_id (внешний ключ на users, уникальный), name (имя), surname (фамилия), about (информация о себе), avatar_url (ссылка на аватар), date (дата рождения);
- roadmaps — дорожные карты: id (первичный ключ), title (название), description (описание), image_url (ссылка на изображение), created_at (дата создания);
- roadmap_steps — этапы (шаги) дорожной карты: id (первичный ключ), roadmap_id (внешний ключ на roadmaps), title (название), description (описание), image_url (ссылка на изображение), order_index (порядковый номер этапа);
- step_links — образовательные ссылки, относящиеся к этапу: id (первичный ключ), step_id (внешний ключ на roadmap_steps), title (название), url (адрес ресурса);
- favorites — избранные пользователем дорожные карты: id (первичный ключ), user_id и roadmap_id (внешние ключи), created_at (дата добавления); пара (user_id, roadmap_id) ограничена условием уникальности;
- user_roadmap_progress — прогресс пользователя по дорожной карте: id (первичный ключ), user_id и roadmap_id (внешние ключи), progress_percent (процент прохождения от 0 до 100), last_step_id (внешний ключ на последний пройденный этап), in_progress (признак активного прохождения), updated_at (дата обновления); пара (user_id, roadmap_id) уникальна;
- user_step_progress — отметки о выполненных пользователем этапах: id (первичный ключ), user_id и step_id (внешние ключи),

is_completed (признак выполнения), completed_at (дата выполнения); пара (user_id, step_id) уникальна.

Связи между таблицами реализованы через механизм внешних ключей. Для большинства связей задано каскадное удаление (ON DELETE CASCADE): при удалении дорожной карты автоматически удаляются её этапы, ссылки на ресурсы, записи об избранном и прогрессе, а при удалении пользователя — его профиль, избранное и записи о прогрессе. Для поля last_step_id таблицы user_roadmap_progress применяется правило ON DELETE SET NULL: при удалении этапа, отмеченного как последний пройденный, соответствующее значение обнуляется без удаления записи о прогрессе. Для ускорения выборки по внешним ключам в таблицах созданы индексы (idx_steps_roadmap, idx_links_step, idx_favorites_user, idx_progress_user, idx_step_progress_user).

2.3.3 Клиентская часть приложения

Клиентская часть приложения реализована на языке Kotlin с использованием декларативного UI-фреймворка Jetpack Compose и построена по архитектурному паттерну MVVM. Каждый экран приложения представлен Composable-функцией, получающей состояние от соответствующей ViewModel через StateFlow, что обеспечивает реактивное обновление интерфейса при изменении данных.

Навигация между экранами реализована с помощью компонента Navigation Compose. Приложение использует паттерн Single Activity, при котором единственная Activity служит контейнером для всех экранов, что соответствует современным рекомендациям Google по архитектуре Android-приложений.[13]

Наиболее технически сложным компонентом клиента является визуализация дорожной карты в виде последовательности этапов. Карта рисуется на Canvas в составе Composable-функции: этапы (milestones) располагаются

вдоль маршрута и соединяются линиями, наглядно отображая последовательность прохождения. Текущий этап выделяется визуально, что позволяет пользователю отслеживать своё местоположение на дорожной карте, а нажатие на этап открывает информационную панель с его описанием.

2.3.4 Взаимодействие клиент-сервер

Взаимодействие между клиентским приложением и сервером построено на основе REST API с использованием протокола HTTPS. Для выполнения сетевых запросов на стороне клиента применяется библиотека Retrofit совместно с HTTP-клиентом OkHttp. Передаваемые данные сериализуются в формат JSON.[14][15]

JWT-токен аутентификации, получаемый при входе в систему, автоматически добавляется в заголовок Authorization каждого защищённого запроса с помощью компонента Interceptor библиотеки OkHttp. Все сетевые операции выполняются асинхронно с использованием корутин Kotlin, что исключает блокировку основного потока приложения и обеспечивает плавный пользовательский интерфейс.

При открытии дорожной карты клиент запрашивает у сервера данные о ней и её этапах, после чего строит визуальную последовательность. Изменения прогресса (начало прохождения карты, отметка этапа выполненным) отправляются на сервер, и состояние дорожной карты обновляется.[15][16]

3 Реализация разрабатываемой системы

3.1 Серверная часть приложения

Серверная часть приложения «Roadmap» реализована в виде самостоятельного Kotlin-проекта на фреймворке Ktor. Точкой входа в серверное приложение является функция `main`, в которой инициализируется встроенный веб-сервер, настраиваются модули приложения и запускается прослушивание входящих соединений на заданном порту. Конфигурация сервера, включая параметры подключения к базе данных и секретный ключ для подписи JWT-токенов, вынесена в файл `application.conf` в формате HOCON.

Подключение к базе данных PostgreSQL реализовано с использованием пула соединений HikariCP, обеспечивающего эффективное переиспользование JDBC-соединений. Схема базы данных создаётся автоматически при первом запуске сервера средствами библиотеки Exposed. При первоначальном развёртывании выполняется наполнение базы данных контентом дорожных карт (seeding): в таблицы загружаются данные о доступных картах и их этапах. [17]

Маршрутизация HTTP-запросов организована в виде отдельных функций-расширений для класса `Route`, группирующих эндпоинты по предметным областям. Защищённые эндпоинты обёрнуты в блок `authenticate`, который автоматически проверяет валидность JWT-токена и возвращает код 401 `Unauthorized` при его отсутствии или невалидности. Все операции с базой данных оборачиваются в транзакции, что обеспечивает атомарность и целостность данных.

Обработка ошибок реализована централизованно с использованием плагина `Ktor StatusPages`, который перехватывает необработанные исключения и возвращает клиенту стандартизированные JSON-ответы с соответствующими HTTP-кодами статуса (400, 401, 404, 500).

3.2 Клиентская часть приложения

Клиентская часть приложения реализована в виде нативного Android-приложения на языке Kotlin с использованием декларативного UI-фреймворка Jetpack Compose. Навигация между экранами организована с помощью Navigation Compose: глобальный навигационный граф (NavHost) управляет последовательностью экранов до входа в систему (загрузка, приветствие, авторизация, регистрация, анкета), а вложенный навигационный граф с нижней панелью навигации (NavigationBar) — основными экранами приложения после авторизации. Каждый экран реализован отдельной Composable-функцией и получает состояние от соответствующей ViewModel. Ниже рассматриваются основные экраны приложения.

3.2.1 Экран загрузки и приветствия

При запуске приложения первым отображается экран загрузки (SplashScreen), на котором демонстрируется логотип приложения. Во время отображения этого экрана выполняется проверка наличия сохранённого токена авторизации: если пользователь уже авторизован, он перенаправляется на главный экран, в противном случае — на экран приветствия. Демонстрация внешнего вида экрана представлена на рисунке 9.



Рисунок 9 - Экран загрузки (SplashScreen)

Экран приветствия (WelcomeScreen) содержит название и краткое описание приложения, а также две кнопки — «Войти» и «Зарегистрироваться», ведущие на соответствующие экраны. Этот экран служит точкой входа для неавторизованного пользователя. Демонстрация внешнего вида экрана представлена на рисунке 10.

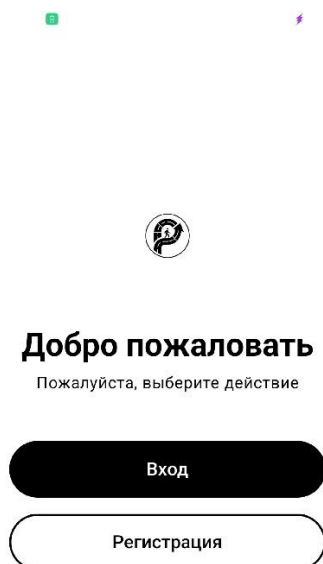
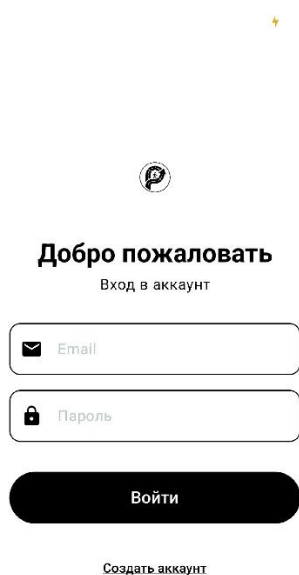


Рисунок 10 - Экран приветствия (WelcomeScreen)

3.2.2 Экран авторизации и регистрации

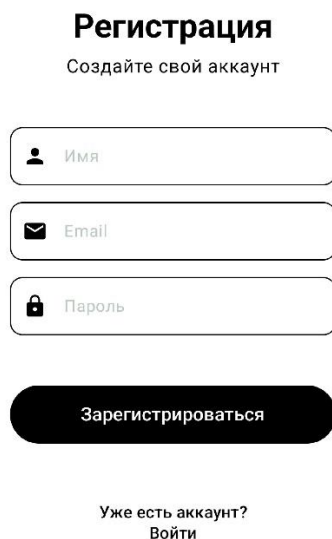
Экран входа (LoginScreen) содержит поля для ввода адреса электронной почты и пароля, а также ссылку на экран регистрации. При нажатии кнопки входа выполняется запрос к серверу; при успешной аутентификации полученный JWT-токен сохраняется в защищённом хранилище средствами TokenManager, и пользователь перенаправляется на главный экран. Обработка состояний загрузки и ошибок осуществляется через LoginViewModel. Демонстрация внешнего вида экрана представлена на рисунке 11.



The image shows a login screen design. At the top center is a small yellow star icon. Below it is a circular logo containing a stylized letter 'P'. The text "Добро пожаловать" (Welcome) is centered below the logo, followed by "Вход в аккаунт" (Login) in a smaller font. There are two input fields: the first is labeled "Email" with an envelope icon, and the second is labeled "Пароль" (Password) with a lock icon. Below these fields is a large black button with the white text "Войти" (Login). At the bottom, there is a link that says "Создать аккаунт" (Create account) in a small, light gray font.

Рисунок 11 - Экран входа (LoginScreen)

Экран регистрации (RegisterScreen) содержит поля для ввода адреса электронной почты и пароля. После успешной регистрации пользователь направляется на экран заполнения анкеты профиля (UserInfoScreen), где указываются имя, фамилия и дата рождения (с выбором даты через DatePicker). После заполнения анкеты пользователь попадает на главный экран приложения. Демонстрация внешнего вида экрана представлена на рисунке 12.



Регистрация
Создайте свой аккаунт

Имя

Email

Пароль

Зарегистрироваться

Уже есть аккаунт?
Войти

Рисунок 12 - Экран регистрации (RegisterScreen)

3.2.3 Главный экран

Главный экран (HomeScreen) открывается после авторизации и является стартовым в нижней панели навигации. На нём отображаются два блока дорожных карт: рекомендованные карты и избранные карты пользователя. Каждая дорожная карта представлена карточкой с изображением, названием и кратким описанием. При выборе карты пользователь переходит к её детальному просмотру, а в случае, если прохождение карты уже начато, на кнопке отображается соответствующая надпись о продолжении. Загрузка данных и обработка ошибок осуществляются через HomeViewModel. Демонстрация внешнего вида экрана представлена на рисунке 13.



Рисунок 13 - Главный экран (HomeScreen)

3.2.4 Экран каталога Дорожных карт

Экран каталога (CatalogScreen) отображает полный список доступных дорожных карт в виде вертикального прокручиваемого списка (LazyColumn). Каждая карта представлена карточкой с изображением, названием и описанием. При выборе дорожной карты происходит переход к экрану её просмотра. Данные каталога предоставляются через CatalogViewModel, который обращается к репозиторию дорожных карт. Демонстрация внешнего вида экрана представлена на рисунке 14.

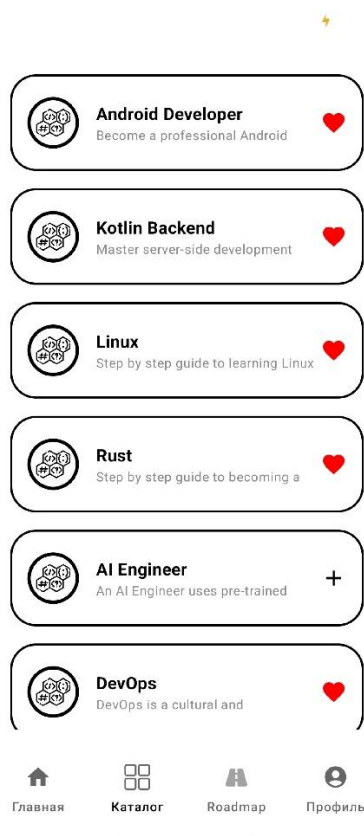


Рисунок 14 - Экран каталога (CatalogScreen)

3.2.5 Экран дорожной карты

Экран дорожной карты (RoadmapScreen) является ключевым и наиболее технически сложным экраном приложения. Он отображает выбранную карту в виде визуальной последовательности этапов (milestones), отрисованной средствами Canvas API Jetpack Compose. Этапы располагаются вдоль маршрута и соединяются линиями, наглядно представляя рекомендуемую последовательность прохождения. Текущий этап выделяется визуально, что позволяет пользователю отслеживать своё местоположение на дорожной карте. Демонстрация внешнего вида экрана представлена на рисунке 15.

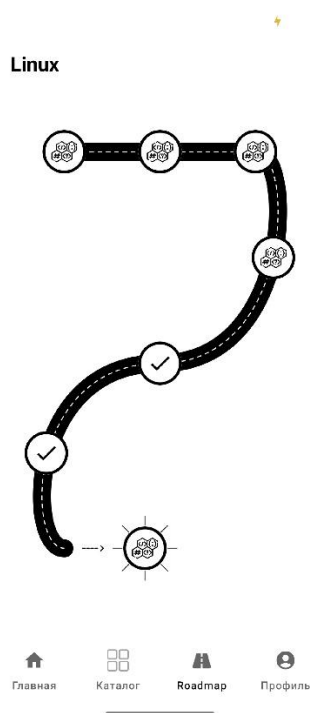


Рисунок 15 - Экран дорожной карты (RoadmapScreen)

При нажатии на этап открывается информационная панель с его названием и описанием. Пользователь может начать прохождение дорожной карты и отмечать этапы как выполненные; при этом прогресс сохраняется на сервере, а индикаторы прохождения обновляются. В случае, если дорожная карта ещё не выбрана, отображается экран-заглушка с предложением перейти в каталог. Логика загрузки данных, подсчёта пройденных этапов и обновления прогресса реализована в RoadmapViewModel.

3.2.6 Экран профиля

Экран профиля (ProfileScreen) предоставляет доступ к персональной информации, статистике обучения и настройкам аккаунта. В верхней части отображается аватар пользователя (или изображение по умолчанию) и его имя. На экране представлены блок текущего прогресса с дорожными картами, прохождение которых начато, и блок избранных карт. Данные профиля и прогресса

предоставляются через ProfileViewModel. Демонстрация внешнего вида экрана представлена на рисунке 16.

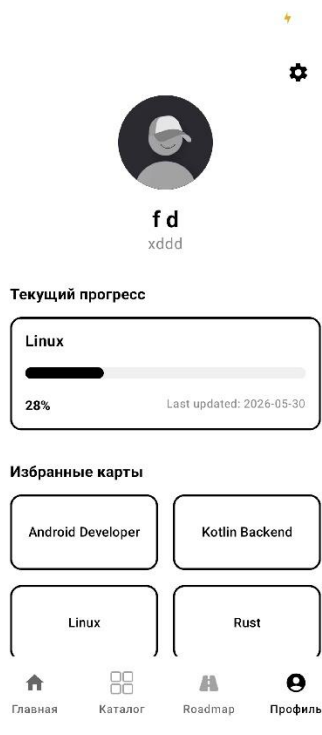


Рисунок 16 - Экран профиля (ProfileScreen)

С этого же экрана пользователь может перейти к редактированию профиля (EditProfileScreen), где доступно изменение персональных данных: имени, фамилии, даты рождения (через DatePicker) и адреса электронной почты, а также загрузка аватара. После сохранения изменения отправляются на сервер через соответствующий эндпоинт. Здесь же расположена функция выхода из аккаунта, которая очищает сохранённый токен и возвращает пользователя к экрану приветствия. Логика редактирования реализована в EditProfileViewModel. Демонстрация внешнего вида экрана представлена на рисунке 17.

Редактирование профиля

Email

Имя

Фамилия

Дата рождения

Должность

Сохранить

Выйти из аккаунта

Главная

Каталог

Roadmap

Профиль

Рисунок 17 - Экран редактирования профиля (EditProfileScreen)

Заключение

В ходе выполнения данной курсовой работы было спроектировано и разработано мобильное Android-приложение «Roadmap», предоставляющее пользователям доступ к структурированным дорожным картам обучения по IT-профессиям и навыкам с возможностью отслеживания персонального прогресса их освоения.

В рамках работы были решены все поставленные задачи. Проведён анализ рынка образовательных IT-приложений и сервисов дорожных карт, в ходе которого рассмотрены три аналога: roadmap.sh, Coursera и Хекслет. Анализ показал, что ни один из них не сочетает структурированные дорожные карты по профессиям и навыкам, визуализацию траектории обучения в виде интерактивного графа, нативное Android-приложение и полноценное отслеживание прогресса, что определило конкурентные преимущества разрабатываемой системы.

Была спроектирована архитектура системы, включающая клиентское Android-приложение, серверную часть на Kotlin/Ktor и реляционную базу данных PostgreSQL. Особое внимание уделено проектированию схемы базы данных, отражающей структуру дорожных карт как графа взаимосвязанных тем. Взаимодействие между компонентами организовано посредством REST API с использованием формата JSON и JWT-аутентификации.

Реализована серверная часть приложения, предоставляющая REST API для получения каталога и содержимого дорожных карт и сохранения персонального прогресса пользователей, а также клиентское Android-приложение с применением Jetpack Compose, включающее полный набор экранов: авторизации и регистрации, каталога дорожных карт, интерактивного графа дорожной карты, темы и профиля с прогрессом.

Разработанное приложение «Roadmap» представляет собой законченный программный продукт, объединяющий достоинства концепции дорожных карт

обучения с удобством нативного мобильного приложения. В качестве направлений дальнейшего совершенствования можно выделить добавление возможности создания пользовательских дорожных карт, внедрение элементов геймификации для повышения мотивации и разработку версии приложения для платформы iOS.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. roadmap.sh: Developer Roadmaps. — Текст: электронный // roadmap.sh: [сайт]. — URL: <https://roadmap.sh> (дата обращения: 30.04.2026).
2. Coursera: Online Courses, Certificates & Degrees. — Текст: электронный // [coursera.org](https://www.coursera.org) : [сайт] –2026. — URL: <https://www.coursera.org> (дата обращения: 13.04.2026).
3. Хекслет: онлайн-школа программирования. — Текст: электронный // ru.hexlet.io: [сайт] – URL: <https://ru.hexlet.io> (дата обращения: 15.04.2026).
4. Kotlin Programming Language: Documentation. — Текст: электронный // kotlinlang.org: [сайт] – URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 16.04.2026).
5. Jetpack Compose UI App Development Toolkit. — Текст: электронный // developer.android.com: [сайт] – URL: <https://developer.android.com/compose> (дата обращения: 20.04.2026).
6. Guide to app architecture. — Текст: электронный // developer.android.com: [сайт] – URL: <https://developer.android.com/topic/architecture> (дата обращения: 23.04.2026).
7. Graphics in Compose. — Текст: электронный // developer.android.com: [сайт] – URL: <https://developer.android.com/develop/ui/compose/graphics/draw/overview> (дата обращения: 24.04.2026).
8. Жемеров, Д. Kotlin в действии / Д. Жемеров, С. Исакова. — М.: ДМК Пресс, 2017. — 402 с.
9. Ktor: Asynchronous Framework for Kotlin. Documentation. — Текст: электронный // ktor.io: [сайт] – URL: <https://ktor.io/docs> (дата обращения: 17.04.2026).
10. PostgreSQL 18 Documentation / The PostgreSQL Global Development Group. — Текст: электронный // [postgresql.org](https://www.postgresql.org): [сайт] – URL: <https://www.postgresql.org/docs/current/index.html> (дата обращения: 27.04.2026).

11. JetBrains Exposed: Kotlin SQL Framework. — Текст: электронный // github.com: [сайт] – URL: <https://github.com/JetBrains/Exposed> (дата обращения: 28.04.2026).
12. Jones, M. JSON Web Token (JWT) / M. Jones, J. Bradley, N. Sakimura. — RFC 7519. — IETF, 2015. — Текст: электронный // URL: <https://data-tracker.ietf.org/doc/html/rfc7519> (дата обращения: 18.05.2026).
13. Navigation with Compose. — Текст: электронный // developer.android.com: [сайт] – URL: <https://developer.android.com/develop/ui/compose/navigation> (дата обращения: 16.05.2026).
14. Retrofit: A type-safe HTTP client for Android and the JVM. — Текст: электронный // square.github.com: [сайт] – URL: <https://square.github.io/retrofit> (дата обращения: 18.04.2026).
15. OkHttp: An HTTP client for Android and the JVM. — Текст: электронный // square.github.com: [сайт] – URL: <https://square.github.io/okhttp> (дата обращения: 18.05.2026).
16. Coroutines: Kotlin Documentation. — Текст: электронный // kotlinlang.org: [сайт] – URL: <https://kotlinlang.org/docs/coroutines-overview.html> (дата обращения: 16.05.2026).
17. HikariCP: A high-performance JDBC connection pool. — Текст: электронный // github.com: [сайт] – URL: <https://github.com/brettwooldridge/HikariCP> (дата обращения: 29.04.2026).